

Uber Lite Ride Sharing System

Project Proposal for UCS503P
Submitted to: Dr. Paramveer Kaur
Thapar Institute of Engineering and Technology

Ayush, CSED
1024030740, aayush_be24@thapar.edu

Gurleen Kaur, CSED
1024030325, gkaur6_be24@thapar.edu

Anshika, CSED
1024030749, aanshika_be24@thapar.edu

August 13, 2026

1 Higher-order goal

... secondary framing

This project aims to improve the accessibility and efficiency of urban ride-sharing by reducing the time, cost, and effort required for users to find and book a suitable ride. The broader goal is to provide a lightweight and practical transportation system that connects passengers with available drivers while maintaining a simple and responsive user experience.

The immediate engineering objective is to translate the core ride-sharing workflow—from requesting a ride to matching a driver and completing the trip—into a measurable and deployable full-stack web application. The system will focus on practical aspects such as location-based driver matching, ride booking, fare estimation, trip-status management, authentication, and reliable communication through REST APIs.

2 Time-to-value

... primary heuristic

- We will deliver meaningful functionality quickly by prototyping the core ride-booking workflow first: user registration, authentication, ride request, driver matching, booking confirmation, and ride-status updates.
- We will prioritize fast feedback over unnecessary complexity by implementing the system in short iterations, validating the main passenger and driver workflows early, and improving the system based on testing results.
- The first iteration will focus on a working end-to-end ride-sharing workflow rather than advanced features. This allows the project to demonstrate measurable value early while leaving room for later improvements such as optimized driver matching, improved database queries, fare adjustments, real-time ride updates, feedback and rating functionality, and support for multiple concurrent ride requests.
- Success will be defined by what can be delivered and measured in the first iteration: a user should be able to authenticate, request a ride, the system should identify an appropriate available driver, and the ride should progress through clearly defined states.

3 Problem Statement

Traditional transportation services can be inconvenient and difficult to manage when users have to search for available vehicles, contact drivers manually, and coordinate ride details through separate channels. Users may also face difficulties in knowing whether a driver is available, tracking the status of a ride request, and estimating the expected fare before or during a trip.

The main problems addressed by UberLite include:

- **Difficulty in finding available drivers:** Users may have to manually search for or contact drivers, leading to delays in booking a ride.
- **Lack of centralized ride management:** Ride requests, driver availability, and ride details are difficult to manage without a unified platform.
- **Limited ride status visibility:** Users may not have clear information about whether a ride is requested, accepted, in progress, or completed.
- **Uncertainty in ride information:** Users may not have a simple way to view important ride details such as pickup location, destination, driver information, and estimated fare.
- **Manual management of ride requests:** Handling bookings and updating ride information manually can result in delays, inconsistencies, and errors.

These issues create the need for a simple and centralized system that can manage the basic interaction between passengers and drivers while providing clear ride information and status updates. UberLite addresses this need through a lightweight digital platform for requesting, matching, and managing rides.

4 Proposed Solution

...Software Proposition

4.1 Overview

Build a lightweight full-stack ride-sharing web application using React, Node.js, Express.js, PostgreSQL, and Prisma. The system will provide a React-based frontend for passengers and drivers and a Node.js and Express.js backend for application logic and REST API communication. The system will provide the following core functionality:

- **User registration and authentication:** Passengers and drivers can register and authenticate before accessing role-specific functionality.
- **Passenger and driver dashboards:** The React frontend will provide role-specific dashboards for passengers and drivers, allowing each user type to access the functions relevant to their workflow.
- **Ride request module:** Passengers can provide pickup and destination locations and submit a ride request through the web application.
- **Driver availability and matching:** The system maintains driver availability and matches ride requests with suitable available drivers based on location.
- **Ride management workflow:** Drivers can accept or reject ride requests and update the ride status throughout the journey.

- **Fare estimation and calculation:** The system calculates an estimated and final fare using predefined pricing rules based on ride-related information.
- **Ride status tracking:** The system maintains the current status of each ride, such as Requested, Accepted, In Progress, or Completed.
- **Ride history:** Completed rides and their relevant details are stored so that previous trips can be retrieved.

The frontend will communicate with the Express.js backend through REST APIs and structured JSON data. PostgreSQL will be used for persistent storage of users, drivers, rides, locations, fare details, and ride history, with Prisma providing the database access layer.

4.2 Core Workflow

1. Passenger registers and authenticates with the system.
2. Passenger submits a ride request containing pickup and destination information.
3. System validates the request and checks for available drivers.
4. System identifies a suitable driver using the driver matching mechanism.
5. Driver accepts or rejects the ride request.
6. Once accepted, the ride status changes from *Requested* to *Accepted*.
7. Driver starts the trip and the ride status changes to *In Progress*.
8. Driver completes the trip and the system calculates the final fare.
9. The completed ride is stored in the ride history.
10. After the ride is completed, the passenger can provide a rating and optional feedback for the driver.

4.3 Operational Constraints for an Educational Setting

- Keep the application lightweight and suitable for deployment on commonly available computing resources.
- Focus on the essential ride-booking workflow rather than replicating the complete functionality of commercial ride-hailing platforms.
- Use a modular full-stack architecture so that frontend, backend, and database components can be tested and extended independently.
- Use mature technologies and libraries rather than developing infrastructure from scratch.
- Avoid unnecessary dependence on complex external services during the initial implementation.
- Prioritize correctness, reliability, efficient data management, and scalability of the core application.

5 Solution Approach

... Engineering focus

This is an engineering course project, so the focus will be on building a reliable, modular, and scalable full-stack application rather than developing novel algorithms. The implementation will prioritize the core passenger-driver workflow, efficient driver matching, well-defined ride-status transitions, secure API access, and incremental development.

5.1 Backend Architecture

... Node.js and Express.js REST-based solution

The system will use a modular Node.js and Express.js backend exposed through REST APIs. The major components will be separated according to their responsibilities:

- **Authentication Module:** Handles user registration, login, JWT-based authentication, and role-based access for passengers and drivers.
- **Ride Management Module:** Creates ride requests, manages ride information, and controls valid ride-status transitions.
- **Driver Management Module:** Maintains driver availability, location information, and active ride assignments.
- **Matching Module:** Searches for suitable available drivers based on their location relative to the pickup point.
- **Fare Module:** Calculates estimated and final fares using predefined pricing rules.
- **Feedback Module:** Records passenger ratings and optional feedback for completed rides and associates them with the relevant driver and ride.
- **Data Management Module:** Handles persistent storage and retrieval of users, drivers, rides, feedback, and ride history using PostgreSQL and Prisma.
- **Validation Module:** Validates incoming API requests before processing them to maintain consistent and reliable application data.

The Express.js backend will expose these functionalities through REST endpoints, with JSON used for structured request and response data.

5.2 Driver Matching

... lightweight matching

Driver matching will initially use a simple location-based approach:

- Maintain the availability and current location of registered drivers.
- When a passenger requests a ride, identify currently available drivers.
- Calculate or estimate the distance between the pickup location and available drivers.
- Prioritize the nearest suitable available driver.
- Update the driver's availability after assignment so that the same driver cannot be assigned to multiple active rides.

The initial implementation will use a straightforward distance-based matching strategy. The matching module will remain independent from the rest of the system so that it can later be optimized using more efficient search or location-based techniques.

5.3 Route and Distance Calculation

...practical location-based computation

The system will use location and distance calculations where required for estimating the distance between relevant locations. The calculated distance can be used by the fare calculation module and can also support driver-selection decisions.

The distance calculation component will remain separate from the ride-management logic so that it can be improved independently as the system grows.

5.4 Ride Workflow Management

...controlled state transitions

The system will manage every ride through a clearly defined workflow:

Requested → Accepted → In Progress → Completed

Only valid state transitions will be permitted. This ensures that ride information remains consistent and prevents invalid operations, such as completing a ride that has not been accepted or started.

5.5 Data Management

...structured and persistent data

The system will maintain structured records for passengers, drivers, rides, and feedback. PostgreSQL will be used as the relational database for persistent storage of users, drivers, rides, locations, fare details, feedback, ratings, and ride history.

Prisma will be used as the database access layer and ORM for defining the application data models and interacting with PostgreSQL. The database layer will be separated from the core ride-management logic so that storage-related changes do not require redesigning the complete application.

Appropriate indexes and relational constraints will be considered for frequently queried and related data such as driver availability, ride status, user roles, and ride history.

5.6 Authentication and Security

...secure API access

Authentication will be implemented so that passengers and drivers can access only the operations relevant to their roles.

JWT-based token authentication will be used for API access. Input validation will be performed before processing requests to prevent invalid or inconsistent data from entering the system.

Zod will be used for request validation, while CORS will be configured to allow controlled communication between the React frontend and the Express.js backend.

5.7 Fare Calculation

...deterministic pricing

The fare calculation will use a predefined pricing model rather than dynamic pricing. A simple model can be represented as:

$$Fare = BaseFare + (Distance \times Rateperkm)$$

The same calculation rules will be applied consistently so that fare estimates and final fares are predictable and easy to test.

6 Evaluation Criteria

... *measurable and attributable*

The success of UberLite will be evaluated using measurable metrics directly related to the core ride-booking workflow. The evaluation will focus on system correctness, efficiency, reliability, application performance, and successful completion of important frontend workflows rather than comparison with commercial ride-hailing platforms.

6.1 Primary Evaluation Metric

... *ride request to driver assignment*

Driver Assignment Time (DAT): The time taken between a passenger submitting a valid ride request and the system assigning an available driver.

- **Target:** The median driver assignment time should remain below a predefined threshold during testing.
- **Measurement:** The metric will be calculated using timestamps recorded when the ride request is created and when a driver is successfully assigned.
- **Attribution:** Both timestamps will be generated and stored by the backend, allowing the metric to be measured directly from application logs and database records.

6.2 Secondary Evaluation Metrics

- **Ride Request Success Rate:** Percentage of valid ride requests that are successfully matched with an available driver.
- **Matching Accuracy:** Percentage of ride requests for which the system selects the intended nearest or most suitable available driver according to the defined matching rule.
- **Fare Calculation Accuracy:** Percentage of test rides for which the calculated fare matches the expected fare according to the predefined pricing formula.
- **Workflow Correctness:** Percentage of tested rides that successfully follow the valid sequence of states from *Requested* to *Accepted*, *In Progress*, and finally *Completed*.
- **API Response Time:** Average time taken by important backend operations such as authentication, ride creation, driver matching, and ride-status updates.
- **Frontend Workflow Completion:** Percentage of test scenarios in which passengers and drivers can successfully complete their intended workflows through the React interface.
- **Feedback Submission Reliability:** Percentage of eligible completed rides for which ratings and feedback can be successfully submitted and stored.
- **Reliability:** Percentage of test scenarios completed without API failures, invalid state transitions, inconsistent ride assignments, or system errors.

6.3 Pilot Validation Plan

... *high-level*

The system will be validated using controlled test scenarios that represent the main passenger and driver workflows.

- Test the system with multiple passenger and driver accounts representing realistic ride-booking scenarios.

- Measure driver assignment time for a set of ride requests under different numbers of available drivers.
- Test cases where drivers accept requests, reject requests, or become unavailable before assignment.
- Verify that invalid ride-status transitions are rejected by the system.
- Compare calculated fares against manually computed expected values using the predefined fare formula.
- Test API endpoints for valid requests, invalid input, and unauthorized access.
- Test the passenger and driver dashboards for successful navigation, authentication, ride-request submission, and ride-status display.
- Test rating and feedback submission after completed rides and verify that the feedback is correctly associated with the relevant ride and driver.

The collected results will be used to identify performance issues, workflow errors, and reliability problems. Improvements will then be made iteratively based on the observed results.

7 Scalability

...with foresight

Although UberLite is initially developed as an academic full-stack application, the system will be designed to handle growth in users, drivers, and ride requests without requiring a complete redesign.

7.1 Efficient Driver Matching

...reducing unnecessary search

Driver matching is the main scalability concern. Instead of repeatedly checking every driver for every ride request, the matching logic will remain modular so that more efficient location-based search techniques can be introduced if the number of drivers increases significantly.

The initial implementation will use a straightforward distance-based matching strategy suitable for a controlled academic environment.

7.2 Data and Backend Scalability

...efficient operations

User, driver, ride, and feedback information will be stored in PostgreSQL with appropriate indexing and relational constraints for frequently queried and related data. The Express.js backend will be structured into separate routes, controllers, validation, business logic, and database operations.

Prisma will provide a consistent database access layer between the backend application and PostgreSQL. This separation will allow individual components to be optimized without requiring major changes to the complete application.

7.3 Scalability Path

...prototype to larger system

The initial system will target a controlled academic environment. For larger deployments, scalability can be improved through more efficient driver-search strategies, optimized PostgreSQL queries and indexes, better handling of concurrent ride requests, pagination for large datasets, and multiple backend instances when required.

8 Technology and Engine Availability

...mature engineering components

UberLite will use a mature set of technologies and libraries that are well suited for implementing a full-stack web application and REST API system. These components allow the project to focus on ride-booking workflow, correctness, scalability, and evaluation rather than developing basic infrastructure from scratch.

- **React:** Used to develop the frontend interface for passengers and drivers and provide role-specific dashboards for the ride-booking workflow.
- **Node.js:** Used as the server-side JavaScript runtime for implementing the backend application.
- **Express.js:** Used for implementing REST APIs and handling HTTP requests, responses, and backend application logic.
- **PostgreSQL:** Used as the relational database for persistent storage of users, drivers, rides, locations, fare details, feedback, ratings, and ride history.
- **Prisma:** Used as the ORM and database access layer for defining application data models and interacting with PostgreSQL from the Node.js backend.
- **Zod:** Used for request-body validation and ensuring that API inputs follow the required structure.
- **JWT:** Used to implement token-based authentication and protect role-specific API endpoints.
- **CORS:** Used to control communication between the React frontend and the Express.js backend.
- **Postman:** Used to test and validate REST API endpoints during development.
- **Git/GitHub:** Used for version control, collaborative development, and maintaining the project repository.

These technologies provide a mature development environment for the project and allow the team to focus on the engineering aspects of UberLite, particularly efficient driver matching, ride workflow management, correctness, testing, and scalability.

9 Project Scope and Deliverables

The project will be developed incrementally, with the first iteration focused on implementing and demonstrating the complete core ride-booking workflow through the backend APIs and web-based frontend.

9.1 Initial Deliverable

...first iteration

- User registration and authentication for passengers and drivers.
- React-based passenger and driver dashboards.
- Ride request creation with pickup and destination information.

- Driver availability management and basic driver matching.
- Ride workflow from *Requested* to *Accepted*, *In Progress*, and *Completed*.
- Basic fare estimation and final fare calculation.
- Location and distance calculation for driver matching and fare estimation.
- Persistent storage of users, drivers, rides, and ride history using PostgreSQL.
- REST API endpoints for the core system functionality.
- Input validation and role-based authentication.
- Unit and integration testing of the main backend modules.

9.2 Subsequent Deliverables

... iteration-friendly

- Improved driver-matching and search efficiency.
- Enhanced handling of concurrent ride requests.
- Improved ride-history search and filtering.
- Rating and feedback functionality for completed rides.
- Real-time ride-status updates between passenger and driver interfaces.
- Periodic driver location updates and improved ride tracking.
- Additional validation, authentication, and security improvements.
- Improved frontend usability and responsive design.
- Performance testing and scalability evaluation.
- Deployment of the application for online demonstration, subject to project time and available resources.

10 Risks and Mitigations

- **Driver matching efficiency:** A simple search may become inefficient as the number of drivers increases. This will be addressed through a modular matching component that can be optimized later.
- **Concurrent ride requests:** Multiple requests may attempt to assign the same driver. Driver availability will be verified before assignment and updated immediately after a successful assignment.
- **Data consistency:** Invalid ride states or inconsistent records may occur if operations are not validated. The system will enforce defined ride-state transitions and validate API inputs. PostgreSQL transactions and relational constraints can also be used where required.
- **Security:** Unauthorized users may attempt to access restricted operations. JWT-based authentication and role-based access control will be applied to relevant API endpoints.

- **Frontend-backend communication:** Incorrect API requests or cross-origin configuration may cause communication failures. REST API contracts, request validation, and appropriate CORS configuration will be used to reduce such issues.
- **Feedback feature complexity:** Rating and feedback functionality will be implemented only for completed rides and kept lightweight so that it does not interfere with the core ride-booking workflow.
- **Real-time feature complexity:** Real-time ride updates and driver-location tracking may require additional implementation effort. These features will be treated as subsequent deliverables and prioritized based on available development time after the core workflow is stable.
- **Evaluation ambiguity:** Performance and correctness can be difficult to assess without clearly defined metrics. Backend timestamps, test cases, and measurable evaluation criteria will be established before final testing.
- **Project complexity:** Adding too many commercial ride-hailing features may make the project difficult to complete. The initial implementation will remain focused on the core ride-booking workflow, with additional features added only when the core system is stable.

11 Summary

UberLite proposes a lightweight full-stack ride-sharing application using React, Node.js, Express.js, PostgreSQL, and Prisma. The system provides the essential workflow of a ride-hailing system, including user authentication, passenger and driver dashboards, ride requests, driver matching, location and distance calculation, fare calculation, ride-status management, ride history, and post-ride feedback.

The project is designed as an engineering-focused solution rather than a research-driven system. It emphasizes time-to-value through incremental development, measurable evaluation through driver assignment time and workflow correctness, and scalability through efficient driver matching, structured relational data management, and modular frontend and backend design.

The initial implementation will provide a complete and testable ride-booking workflow through a React frontend and Node.js/Express.js backend, with PostgreSQL providing persistent data storage and Prisma providing the database access layer. Subsequent development may extend the system with rating and feedback functionality, real-time ride-status updates, driver location tracking, improved frontend interaction, and online deployment depending on the available project time and testing results.